

DK LEARNING

# GRADE 9

## Programming Systems

Level: **Upper Advanced** • Purpose: Learn before you practise

[← BACK TO LEARNING WEBSITE](#)

### In this guide

1. Data Structures
2. Input Validation
3. Debugging
4. Testing Programs
5. Security and Privacy in Code
6. Practice questions, answers and glossary



## How to use this guide

Read each chapter slowly. Look at the example, then try the “Your turn” task **without copying**. Write your own notes on the lines. When you finish, answer the practice questions and check your work with the answer page. After that, open the Grade 9 activity on the website.

### Learning goals

- Choose lists and dictionaries for data
- Validate input to stop errors
- Debug programs using error messages and tracing
- Think about security and privacy in programs

### Study plan

| Step | What to do                                        |
|------|---------------------------------------------------|
| 1    | Read the chapter and highlight key words          |
| 2    | Study the worked example                          |
| 3    | Do the “Your turn” task on paper or on a computer |
| 4    | Write a short summary in your own words           |
| 5    | Answer the practice questions, then check         |

**Tip:** Never trust user input. Always ask: what is the worst value someone could type here?

# 1 Data Structures

Data structures organise information so programs can work with it efficiently. Lists are useful for ordered collections; dictionaries can connect keys to values.

Choose the structure that fits: a list of marks, a dictionary of student names and marks.

## Key points

- List = ordered items
- Dictionary = key and value pairs
- Access dictionary by key
- Choose the structure for the job

## Worked example

```
student = {"name": "Kasun", "grade": 9}
print(student["name"])
student["grade"] = 10
```

## Your turn

Create a dictionary for a book with title, author and pages.

## Check yourself

Explain each term in your own words:

### Dictionary

---

### Key

---

---

## My notes

---

---

---

---

---

## 2 Input Validation

Validation checks whether input is acceptable before the program uses it. For example, a mark should normally be within an expected range. Types of check include range, type, length and presence checks.

Validation reduces errors and helps stop misuse.

### Key points

- Range check
- Type check
- Length check
- Presence check

### Worked example

```
mark = int(input("Mark: "))
while mark < 0 or mark > 100:
    mark = int(input("Enter 0-100: "))
```

### Your turn

Write a validation loop for a password with at least 8 characters.

### Check yourself

Explain each term in your own words:

#### Validation

---

#### Verification

---

---

### My notes

---

---

---

---

---

## 3 Debugging

Debugging is the process of locating and fixing defects (bugs). Read the error message, find the line number, then trace the values step by step. Common errors: syntax errors, runtime errors and logic errors.

A logic error runs but gives a wrong answer. Use `print()` to see values while running.

### Key points

- Syntax error: wrong grammar
- Runtime error: crashes while running
- Logic error: wrong result
- Trace with `print()`

### Worked example

```
total = 0
for n in [2, 4, 6]:
    total = total + n
print("Average:", total / 2)    # bug: should be / 3
```

### Your turn

Find the bug: for `i` in `range(1, 5)` should print 1 to 5.

### Check yourself

Explain each term in your own words:

#### Bug

---

#### Trace

---

---

### My notes

---

---

---

---

---

## 4 Testing Programs

Testing checks a program works. Use normal data, boundary data (the edges, like 0 and 100) and invalid data (like -5 or text). Write expected results first and compare with actual results.

Good tests find mistakes before users do.

### Key points

- Normal, boundary, invalid data
- Write expected results
- Compare with actual
- Retest after fixing

### Worked example

Test table for a mark program:

50 -> Pass (boundary)

49 -> Fail (boundary)

-5 -> Error (invalid)

### Your turn

Make a test table with six tests for a pass/fail program.

### Check yourself

Explain each term in your own words:

#### Boundary data

---

#### Expected result

---

---

### My notes

---

---

---

---

## 5 Security and Privacy in Code

Developers think about security, privacy and unexpected inputs. Never store passwords as plain text; store hashed values. Do not collect more personal data than needed.

Programs should handle errors politely and not reveal private details.

### Key points

- Do not store plain passwords
- Collect only needed data
- Handle errors safely
- Keep software updated

### Worked example

Bad: users.txt contains "kasun,12345"

Better: store a hash of the password.

### Your turn

List three personal data items a school app should NOT collect.

### Check yourself

Explain each term in your own words:

#### Hash

---

#### Privacy

---

---

### My notes

---

---

---

---

---



## Practice Questions

### Part A - Choose the best answer

1. A dictionary stores...
  - A) key-value pairs
  - B) only numbers
  - C) pictures
2. A range check tests...
  - A) a value is in limits
  - B) text length
  - C) file type
3. A program runs but gives wrong results. This is a...
  - A) logic error
  - B) syntax error
  - C) power error
4. Boundary data is...
  - A) at the edge of limits
  - B) random text
  - C) empty only
5. Passwords should be stored as...
  - A) hashes
  - B) plain text
  - C) photos
6. Which finds the line of a runtime error?
  - A) The error message
  - B) The mouse
  - C) The monitor

### Part B - Short answers

1. What is validation?

---

---

2. Give one boundary value for a mark between 0 and 100.

---

---

**3.** What is a logic error?

---

---

**4.** Why not store plain passwords?

---

---



# Answer Page

## Part A

1-A 2-A 3-A 4-A 5-A 6-A

## Part B (sample answers)

1. Checking input is acceptable before use.
2. 0 or 100 (also 1, 99).
3. The program runs but gives incorrect results.
4. If stolen, anyone can read them.

## Mini glossary

|                        |                                         |
|------------------------|-----------------------------------------|
| <b>Dictionary</b>      | Data stored as key: value pairs         |
| <b>Key</b>             | The label used to find a value          |
| <b>Validation</b>      | Checking input is sensible              |
| <b>Verification</b>    | Checking data was entered correctly     |
| <b>Bug</b>             | A defect in a program                   |
| <b>Trace</b>           | Following values line by line           |
| <b>Boundary data</b>   | Values at the edge of the allowed range |
| <b>Expected result</b> | What should happen                      |
| <b>Hash</b>            | A one-way scrambled value of data       |
| <b>Privacy</b>         | Control over personal information       |



## Activity Preparation

Solve validation and debugging scenarios and produce a short program design. Then open the Grade 9 activity.

### Think before you start

- Explain one important idea from this grade in your own words.
- Give one real-life example where this topic is useful.
- What mistake could a beginner make, and how can it be prevented?
- Write the steps you will follow before starting the Grade 9 activity.

### Ready checklist

- |                                                     |                                                        |
|-----------------------------------------------------|--------------------------------------------------------|
| <input type="checkbox"/> I read every chapter       | <input type="checkbox"/> I tried the “Your turn” tasks |
| <input type="checkbox"/> I understand the key terms | <input type="checkbox"/> I checked my practice answers |
| <input type="checkbox"/> I wrote my own notes       | <input type="checkbox"/> I am ready for the activity   |

⚡ **START GRADE 9 ACTIVITY**

← **EXIT PDF / BACK TO LEARNING**